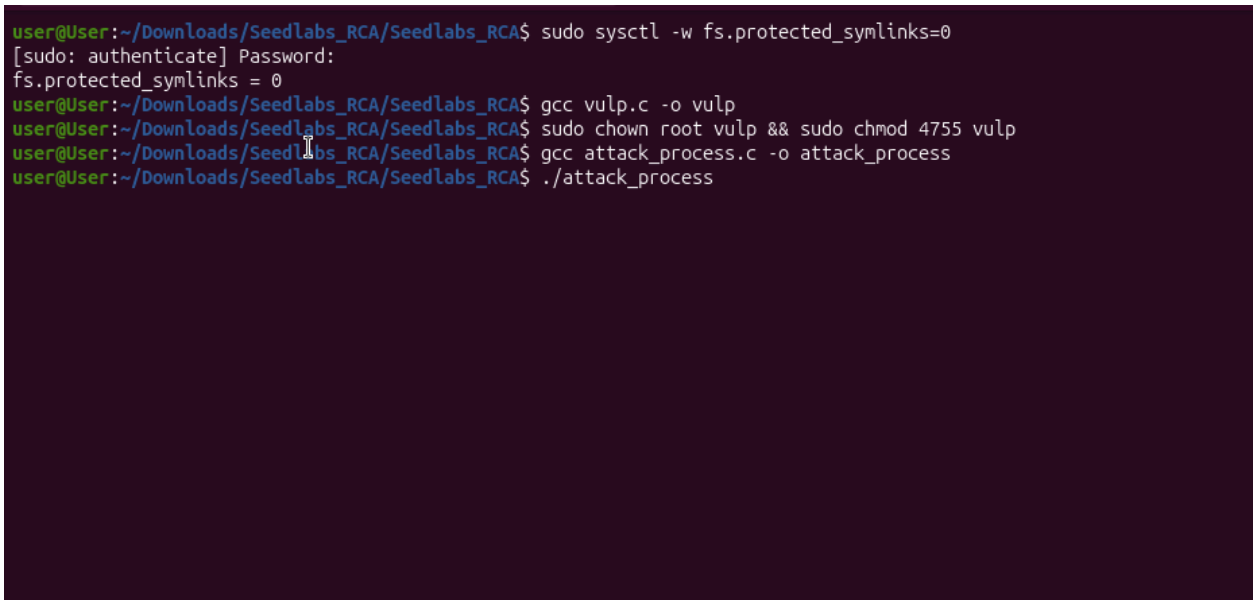
CYSE 211 - Lab 5

Harshith Motipalli
Professor Gebril

Purpose of Lab:

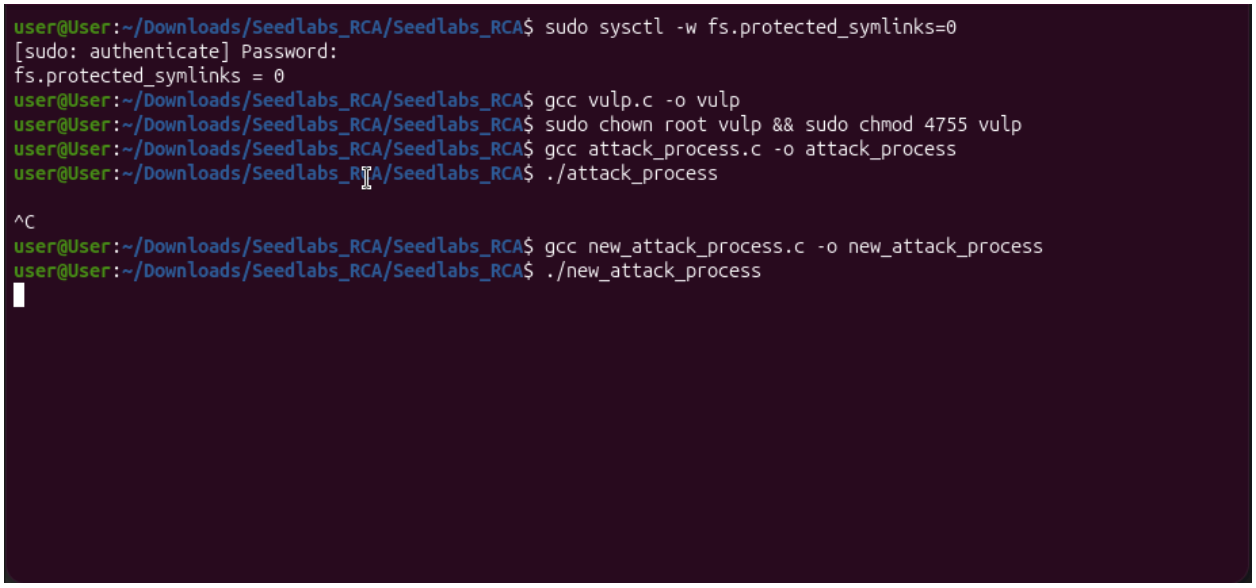
The main goal of this lab is to get hands-on experience exploiting a race-condition vulnerability. We do this by running an attack script at the exact same time as a vulnerable SetUID program (vulp) to trigger a time-of-check to time-of-use (TOCTOU) flaw. By rapidly swapping out a symbolic link during that tiny window of execution, we can trick the program into writing a malicious payload into a protected system file. Ultimately, this allows us to create a new user account named "test" that has a blank password and full root privileges.

Step 1:



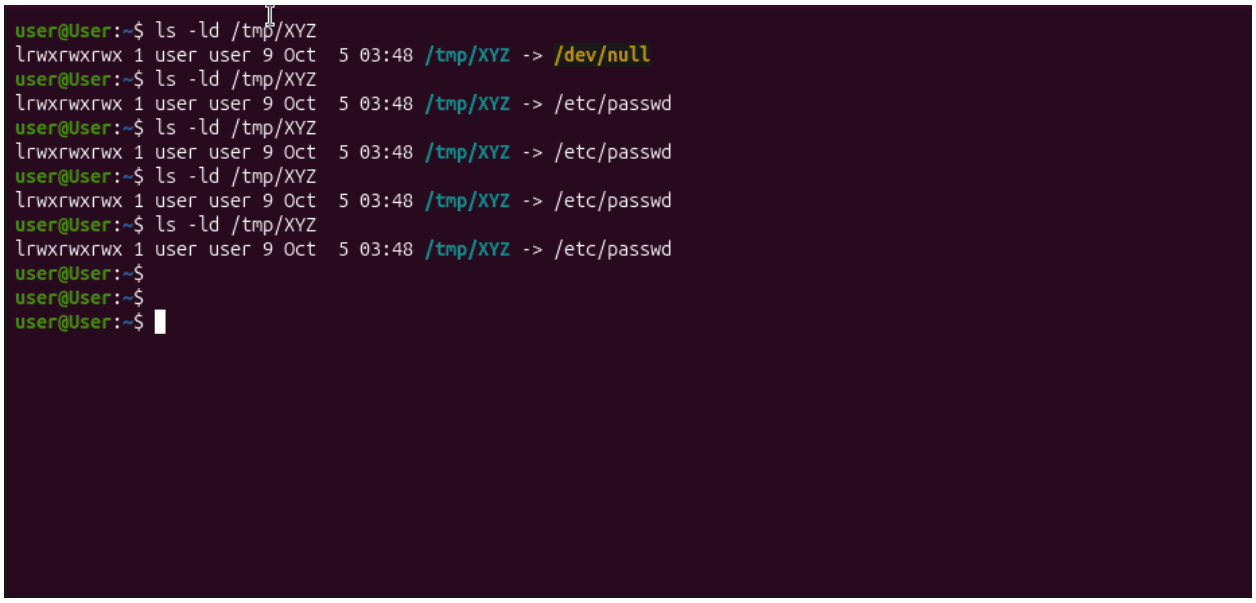
Description: First, we set up the environment by disabling the system's built-in symbolic link protections using sysctl, which makes the attack possible. Next, we compile our target program (vulp.c) and change its permissions to make it a SetUID executable owned by root (chmod 4755). This step is crucial because it ensures the vulnerable program runs with root privileges, no matter who actually launches it.

Step 2:



Description: In this step, we pivot to using the corrected attack code after the initial attempt fails. Because the original attack_process.c file contains its own race condition vulnerability, it is highly unreliable and often causes the exploit to get stuck. To resolve this, we interrupt the stalled attack process (indicated by the ^C command) and compile the provided new_attack_process.c file instead. Finally, we execute this newly compiled, optimized attack process in the background, allowing us to reliably trigger the time-of-check to time-of-use (TOCTOU) exploit without it failing.

Step 3:



Description: By running the ls -ld /tmp/XYZ command multiple times, we can actually see the attack process rapidly changing the /tmp/XYZ symbolic link in real-time. The link constantly flips back and forth between pointing to /dev/null and pointing to the highly sensitive /etc/passwd file. This rapid swapping is the core of the attack: it is trying to trick the vulnerable program right in the split second after it checks file permissions but before it actually writes the data.

Step 4:

```
lrwxrwxrwx 1 user user 9 Oct  5 03:48 /tmp/XYZ -> /etc/passwd
user@User:~$ ls -ld /tmp/XYZ
lrwxrwxrwx 1 user user 9 Oct  5 03:48 /tmp/XYZ -> /etc/passwd
user@User:~$ ls -ld /tmp/XYZ
lrwxrwxrwx 1 user user 9 Oct  5 03:48 /tmp/XYZ -> /etc/passwd
user@User:~$ ls -ld /tmp/XYZ
lrwxrwxrwx 1 user user 9 Oct  5 03:48 /tmp/XYZ -> /etc/passwd
user@User:~$
user@User:~$
user@User:~$ su test
su: user test does not exist or the user entry does not contain all the required fields
user@User:~$ tail /etc/passwd
geoclue:x:109:113::/var/lib/geoclue:/usr/sbin/nologin
cups-browsed:x:110:111::/nonexistent:/usr/sbin/nologin
pipewire:x:981:981:system user for pipewire:/nonexistent:/usr/sbin/nologin
hplip:x:111:7:HPLIP system user:/run/hplip:/bin/false
gnome-remote-desktop:x:980:980:GNOME Remote Desktop:/var/lib/gnome-remote-desktop:/usr/sbin/nologin
polkitd:x:979:979:User for polkitd:/:/usr/sbin/nologin
rtkit:x:978:978:RealtimeKit:/proc:/usr/sbin/nologin
colord:x:977:977:colord colour management daemon:/var/lib/colord:/usr/sbin/nologin
gdm:x:975:975:Gnome Display Manager:/var/lib/gdm3:/bin/false
user:x:1000:1000:User:/home/user:/bin/bash
user@User:~$
```

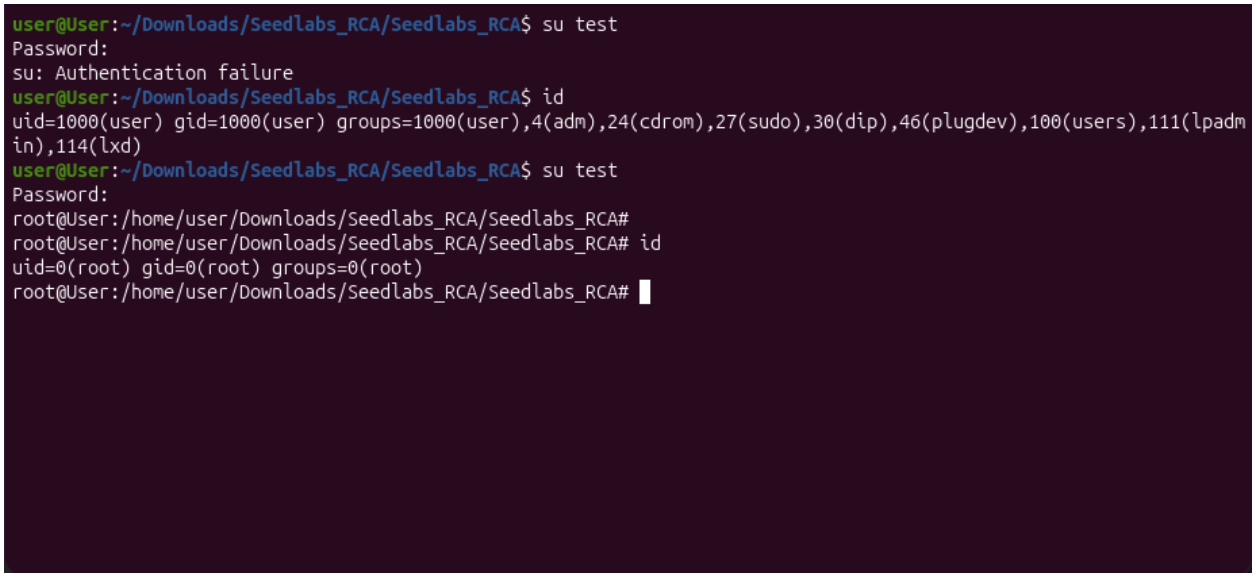
Description: Before triggering the exploit, we need to establish a baseline to prove the system has not been compromised yet. When we try to switch to the "test" user using su test, the system throws an error, confirming that the account does not exist. Checking the bottom of the password file with tail /etc/passwd also backs this up, showing just the normal system users and proving our malicious payload has not been injected yet.

Step 5:

```
user@User:~/Downloads/Seedlabs_RCA/Seedlabs_RCA$ ./target_process.sh
No permission
No permission
No permission
No permission
No permission
No permission
No permission
No permission
No permission
No permission
No permission
No permission
No permission
No permission
No permission
No permission
No permission
No permission
No permission
No permission
No permission
No permission
No permission
No permission
No permission
```

Description: Next, we run the ./target_process.sh script to force the vulnerable SetUID program to start interacting with our rapidly changing symbolic link. The terminal immediately floods with "No permission" messages. This happens because the script is aggressively looping the vulnerable program, trying over and over again to hit the exact fraction of a second needed to win the race condition and bypass the permission check.

Step 6:



Description: Our first attempt to log in with su test fails with an "Authentication failure," and checking our user ID confirms we are still just a standard user (uid=1000). This perfectly illustrates how unreliable the original attack code is, as it often ends up corrupting the payload or locking the symlink entirely. However, once we clear the locked files and run the corrected attack code, running su test works perfectly. It bypasses the password prompt and drops us directly into a root shell, which is officially confirmed by the # prompt and the id command showing uid=0(root).